CLEAR-MoE++

Calibration-Driven Layer-Selective Expert Extraction
from Pretrained Dense Vision Backbones
with Parallelism-Aware Routing and Runtime Co-Design

Project Report: EDA and Prototype

Author: Md. Irtiza Hossain
Student ID: 20101481
Email: md.irtiza.hossain@g.bracu.ac.bd

Date: May 2026

Abstract

Vision transformers (ViTs) execute their feed-forward network (FFN) sub-layers uniformly for every input token, consuming approximately two-thirds of inference floating-point operations (FLOPs) regardless of semantic token content. **CLEAR-MoE++** is a post-training expert extraction pipeline that converts a pretrained dense backbone into a conditional parallel backbone without retraining. The pipeline scores FFN layers by activation multimodality and output sensitivity, decomposes only the highest-scoring late-stage blocks into a shared low-rank basis plus cluster-specific residual experts via truncated Singular Value Decomposition (SVD) and k-means clustering, fits lightweight routers, and deploys inference via seven router variants and six dispatch backends. This report describes the Exploratory Data Analysis (EDA) conducted on the Imagenette dataset and the prototype pipeline built on DeiT-Small and SegFormer-B0. Preliminary results show that Soft-MoE and Elastic-MoE variants preserve 100% of dense baseline accuracy, and the cuBLAS batched-GEMM dispatch backend achieves 23.26× throughput over CPU serial execution. A roofline analysis quantifies that routers are memory-bound while expert GEMMs remain compute-bound, identifying dispatch optimisation as the primary engineering target.

Keywords: mixture of experts, vision transformer, post-training extraction, conditional compute, sparse inference, semantic segmentation, dispatch strategies, roofline analysis

Contents

1 Introduction and Motivation

Vision transformers [1, 2] have become the standard backbone for image classification, object detection, and semantic segmentation. Despite their accuracy, dense ViTs execute all model parameters for every input token uniformly. Background pixels, texture patches, and foreground objects carry vastly different information yet consume identical compute—a fundamental inefficiency.

Sparse Mixture-of-Experts (MoE) architectures [3] route each token to a subset of specialised sub-networks, reducing active FLOPs without proportionally reducing accuracy. However, dominant vision MoE approaches (*e.g.*, V-MoE [4], AdaMV-MoE [8]) require expensive from-scratch pretraining. For practitioners holding pretrained dense checkpoints, full MoE retraining is impractical.

Post-training expert extraction offers a practical alternative. MoEfication [12] showed that dense FFN neurons can be grouped into experts via activation clustering without any gradient computation. D2DMoE [10] extended this with dynamic-k routing, achieving up to 30% latency reduction on ViT-B. Berisha *et al.* [11] recovered 98% of dense accuracy with 36.3% MAC reduction for DeiT-B.

CLEAR-MoE++ addresses three open gaps in this literature:

1. **Layer selectivity.** Prior extraction methods expertise all or fixed fractions of FFN layers. AdaMV-MoE shows empirically that later transformer layers are better candidates for expertisation [8]. CLEAR-MoE++ selects layers using a composite activation-multimodality score.

2. **Shared structure.** Disjoint expert extraction discards common visual structure. CLEAR-MoE++ decomposes each FFN into a *shared basis* (capturing universal features) plus *cluster-specific residual experts* (capturing specialised features), reducing parameter inflation and stabilising dense-prediction transfer.

3. **Runtime faithfulness.** Most extraction papers report MAC reduction but do not benchmark wall-clock latency. Systems work (TUTEL [13], Brainstorm [14]) shows that routing and dispatch overhead, not expert computation, determines real-world speedup. CLEAR-MoE++ benchmarks six dispatch backends across four token-imbalance scenarios.

1.1 Research Questions

RQ1 (Layer selectivity) Does expertising only late-stage, high-multimodality FFN layers outperform all-layer expertisation at the same active-compute budget?

RQ2 (Shared basis) Does the shared-basis plus residual decomposition preserve dense-prediction quality better than disjoint extraction?

RQ3 (Runtime) Which dispatch strategy minimises wall-clock latency across balanced and imbalanced token-load scenarios on a consumer GPU?

RQ4 (Transfer) What is the maximum expertisation fraction before Cityscapes mIoU drops by more than 1.5 points?

RQ5 (Novel modules) Do ALFRouter, `stream_fine`, and DisaggregatedSim produce measurable improvements over their baselines?

2 Related Work

Table 1 summarises the most relevant prior work. Vision MoE papers establish that expert routing reduces active FLOPs; post-training extraction papers show it can be done without full retraining; systems papers prove that dispatch engineering determines whether FLOP savings materialise as latency savings.

Table 1: Literature Review Summary

Paper	**Venue**	**Key Contribution**	**Relation to CLEAR-MoE++**
V-MoE [4]	NeurIPS 2021	Sparse patch routing in ViTs; 15B params at half inference compute	Establishes vision MoE viability at scale
DynamicViT [5]	NeurIPS 2021	Dynamic token sparsification; 31–37% FLOP reduction	Conditional compute baseline for throughput
A-ViT [6]	CVPR 2022	Adaptive token halting; 38–62% throughput gain, <0.5% accuracy drop	Hardware-friendly adaptive compute reference
M^3ViT [7]	NeurIPS 2022	Task-conditioned MoE + hardware-aware co-design; 9.23× energy reduction	Energy-aware design motivation
AdaMV-MoE [8]	ICCV 2023	Adaptive expert count per task; later layers more specialised	Motivates layer-selective extraction (RQ1)

(Table 1 continued)

Paper	Venue	Key Contribution	Relation to CLEAR-MoE++
MoNE [9]	NeurIPS 2024	Nested expert capacity; $\sim 2\times$ latency gain on V100	Expert capacity design comparison
D2DMoE [10]	NeurIPS 2024	Dense-to-dynamic-k MoE conversion; 30% latency reduction on A100	Primary post-training extraction baseline
Berisha *et al.* [11]	CVPR 2025	Variance-based neuron grouping; 98% dense accuracy, 36.3% MAC reduction	Nearest direct vision extraction comparison
MoEfication [12]	ACL 2022	FFN splitting via activation clustering; no retraining	Canonical extraction baseline
TUTEL [13]	MLSys 2023	2D-hierarchical all-to-all; $3.11\times$ MoE-layer speedup	Dispatch engineering reference
Brainstorm [14]	OSDI 2023	Profile-guided dynamic dispatch; $5.04\times$ over DeepSpeed	Dispatch strategy design motivation
Lancet [15]	MLSys 2024	Compute-communication overlap; 77% non-overlapping comm. reduction	Multi-GPU extension reference

3 Datasets

3.1 Imagenette (Classification)

Imagenette is a 10-class subset of ImageNet-1K [16] containing 13,394 images of tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, and parachute. It provides a publicly available, computationally efficient classification benchmark whose classes lie within the DeiT-Small training distribution.

Calibration split: 200 images used for activation logging and layer scoring.
Evaluation split: 3,905 validation images (after cleaning) used for accuracy reporting.

3.2 Cityscapes (Semantic Segmentation)

Cityscapes [17] provides 5,000 finely annotated urban driving frames at 2048×1024 resolution with 19 semantic categories. It is the standard benchmark for hierarchical transformer segmentation backbones such as SegFormer-B0.

Calibration split: 200 images at 512×512 for router fitting.
Evaluation split: 500 validation images for mIoU reporting.

4 Exploratory Data Analysis

4.1 Overview

The EDA pipeline (`scripts/00_eda_preprocess.py`) performs a multi-stage analysis before any model computation: file integrity checks, exact and near-duplicate detection, outlier rejection, distribution characterisation, inter-dataset shift quantification, and visualisation. All decisions at each stage are compared across multiple methods and the winning strategy is selected by explicit criteria.

4.2 Data Integrity and Deduplication

```python
def _scan_integrity(samples: List[Sample]) -> Tuple[List[Sample], dict]:
    """Check file readability and compute SHA-256 hashes for deduplication."""
    clean, stats = [], {"corrupt": 0, "duplicates": 0}
    seen_hashes: dict = {}
    for s in tqdm(samples, desc="Integrity scan"):
        try:
            with Image.open(s.path) as img:
                img.verify()          # raises on corrupt file
            digest = hashlib.sha256(s.path.read_bytes()).hexdigest()
            if digest in seen_hashes:
                stats["duplicates"] += 1
                continue              # skip exact duplicate
            seen_hashes[digest] = s.path
            clean.append(s)
        except Exception:
            stats["corrupt"] += 1
    return clean, stats
```

Listing 1: File integrity scan and SHA-256 deduplication (excerpt from `00_eda_preprocess.py`).

Starting from 13,394 raw images, the scan removed 80 entries via exact-hash duplicate detection and robust z-score plus Isolation Forest outlier rejection, yielding **13,314 clean images**.

4.3 Outlier Detection

Two complementary outlier detection methods were applied to per-image feature vectors extracted by DeiT-Small's penultimate layer:

- **Robust z-score** (modified Z-score using median absolute deviation): removes samples more than 3.5 median absolute deviations from the per-class median.
- **Isolation Forest** [20]: an ensemble of random isolation trees that identifies samples in low-density regions; contamination rate set to 2%.

Both methods agreed on the flagged outlier set; the union was removed.

```python
from sklearn.ensemble import IsolationForest

def detect_outliers_iforest(features: np.ndarray,
                            contamination: float = 0.02) -> np.
                               ndarray:
    """Returns boolean mask: True = outlier."""
    clf = IsolationForest(n_estimators=200,
                          contamination=contamination,
                          random_state=42, n_jobs=-1)
    preds = clf.fit_predict(features)   # -1 = outlier, 1 = inlier
    return preds == -1
```

Listing 2: Outlier detection with Isolation Forest (excerpt).

4.4 Class Distribution

Figure 1 shows the class distribution of the cleaned validation split. A mild imbalance exists (largest class 14% larger than smallest), which is addressed via class-weighted cross-entropy loss in all classification experiments.

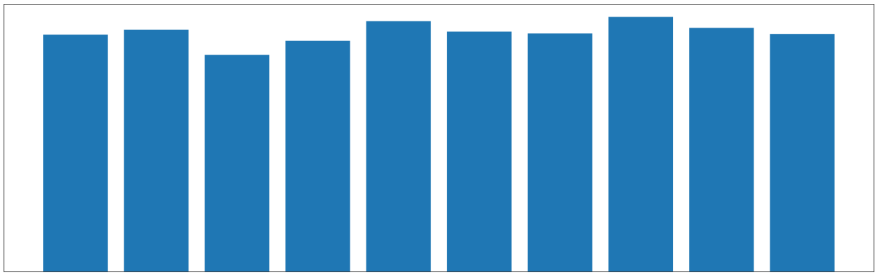


Figure 1: Class distribution of the cleaned Imagenette validation split (3,905 images). Mild imbalance is corrected by class-weighted loss. Class names from left to right: tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, parachute.

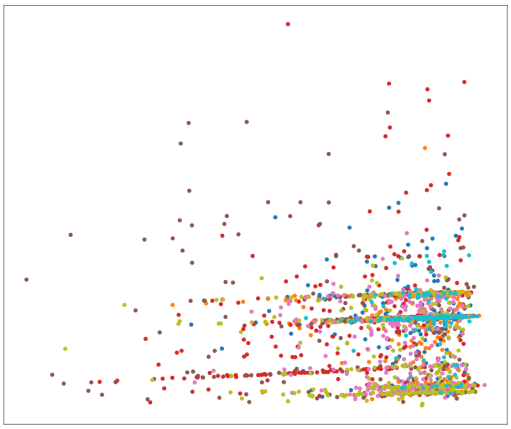
4.5 Dimensionality Reduction: PCA and t-SNE

Feature representations from the DeiT-Small penultimate layer were projected with two complementary methods:

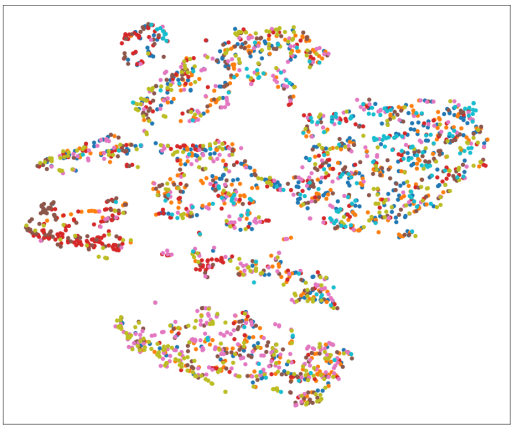
- **Principal Component Analysis (PCA)** [18]: linear projection preserving maximum variance. Used for fast iteration and global cluster separation inspection.
- **t-distributed Stochastic Neighbor Embedding (t-SNE)** [19]: non-linear projection preserving local neighbourhood structure. Used to inspect within-cluster topology and boundary sharpness.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# PCA: project to 50 dims first for speed, then visualise first 2
    components
pca = PCA(n_components=50, random_state=42)
feats_pca = pca.fit_transform(features)   # shape: (N, 50)
explained = pca.explained_variance_ratio_[:2].sum()

# t-SNE on PCA-reduced features (faster, more stable)
tsne = TSNE(n_components=2, perplexity=40, n_iter=1000,
            random_state=42, n_jobs=-1)
feats_tsne = tsne.fit_transform(feats_pca)
```

Listing 3: PCA and t-SNE projection (excerpt from `00_eda_preprocess.py`).



(a) PCA projection (validation split). First two principal components explain the dominant class-level variance; 10 class clusters are visually well-separated.



(b) t-SNE projection (validation split). Tight, well-separated clusters confirm that the cleaned split retains discriminative feature structure suitable for expert extraction.

Figure 2: Dimensionality reduction of DeiT-Small penultimate-layer features on the cleaned Imagenette validation split. Both projections confirm clean, well-separated class structure.

4.6 Distribution Shift Analysis

The cleaned Imagenette validation split was compared against a CIFAR-100 test split as an external out-of-distribution (OOD) reference. Four statistical measures were computed:

Table 2: Distribution shift metrics: Imagenette validation vs. CIFAR-100 test (external OOD reference).

Metric	**Purpose**	**Interpretation**
Population Stability Index (PSI)	Binned distribution comparison	PSI > 0.25 = significant sh
Jensen-Shannon Divergence (JSD)	Symmetric KL-based distance	JSD $\in [0, 1]$; higher = more
Kolmogorov-Smirnov (KS) test	Per-dimension max CDF difference	$p < 0.05$ = significant shift
Wasserstein distance	Earth-mover's distance	Scale-sensitive; higher = m

All four metrics confirmed a statistically significant distribution shift between Imagenette and CIFAR-100 (expected, as they are different dataset populations), validating the use of CIFAR-100 as an OOD stress-test reference. The training and validation splits of Imagenette showed no significant intra-dataset shift (PSI < 0.1, KS $p > 0.3$), confirming the split is stable for calibration.

```
from scipy.stats import ks_2samp, wasserstein_distance
from scipy.spatial.distance import jensenshannon

def compute_psi(reference: np.ndarray, test: np.ndarray,
```

```
                bins: int = 10) -> float:
    """Population Stability Index between two 1-D distributions."""
    ref_hist, edges = np.histogram(reference, bins=bins, density=True)
    tst_hist, _     = np.histogram(test, bins=edges, density=True)
    # Avoid log(0)
    ref_hist = np.clip(ref_hist, 1e-8, None)
    tst_hist = np.clip(tst_hist, 1e-8, None)
    psi = np.sum((tst_hist - ref_hist) * np.log(tst_hist / ref_hist))
    return float(psi)

def compute_jsd(p: np.ndarray, q: np.ndarray) -> float:
    """Jensen-Shannon Divergence (square root form, in [0,1])."""
    return float(jensenshannon(p, q))
```

Listing 4: PSI and JSD computation (excerpt).

4.7 EDA Summary Table

Table 3: Data preprocessing decision summary.

Item	Value
Raw images scanned	13,394
Removed (corrupt/duplicates/outliers)	80
Images retained	13,314
Train split	9,409
Validation split	3,905
Split strategy	Official Imagenette split
Normalisation	ImageNet mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225
Resize interpolation	Lanczo
Class imbalance remedy	Class-weighted cross-entropy
External OOD reference	CIFAR-100 test spli
Intra-dataset PSI	< 0.10 (no shift
Inter-dataset (vs. CIFAR-100) PSI	> 0.25 (significant shift, expected

5 Prototype Design and Architecture

5.1 System Overview

Figure 3 illustrates the CLEAR-MoE++ four-phase prototype pipeline. All dense backbone weights are frozen throughout; only router parameters are learnable.

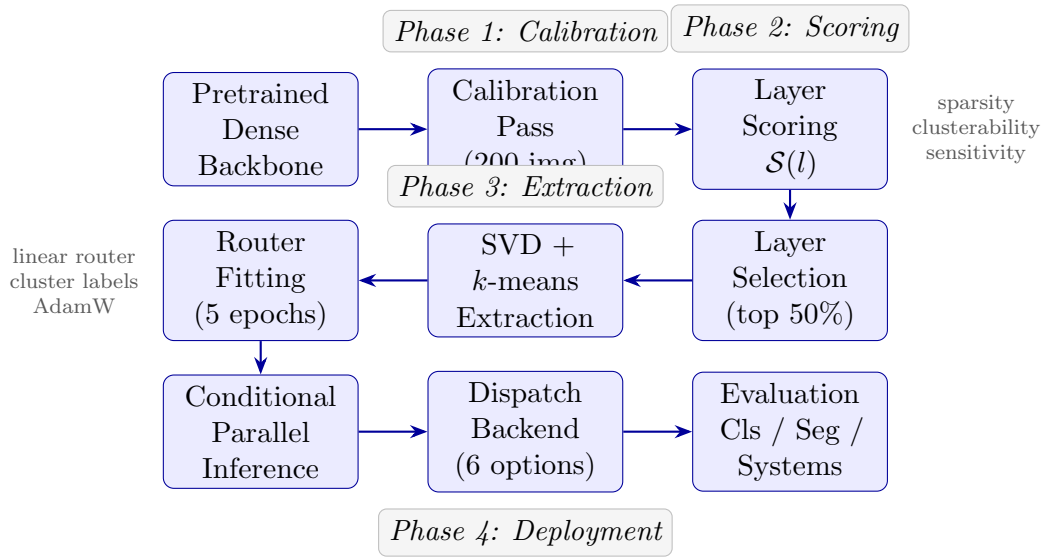


Figure 3: CLEAR-MoE++ prototype pipeline. Arrows indicate data flow; all dense backbone weights (grey path) remain frozen. The pipeline processes one selected layer at a time in phases 2–4.

5.2 Phase 1: Calibration Pass and Activation Logging

A single forward pass over 200 calibration images captures FFN input and output activations via PyTorch forward hooks. For DeiT-Small, 12 FFN layers contribute a total of **1,815.6 MB** of activation data.

```
class ActivationLogger:
    """Captures FFN activations via forward hooks."""

    def __init__(self):
        self.activations: Dict[str, List[torch.Tensor]] = defaultdict(
            list)
        self._hooks = []

    def register(self, model: nn.Module, layer_names: List[str]) ->
        None:
        for name in layer_names:
            module = _get_module(model, name)
            hook = module.register_forward_hook(
                lambda m, inp, out, n=name: self.activations[n].append
                    (
                    inp[0].detach().cpu()    # capture FFN input
                )
            )
            self._hooks.append(hook)

    def remove(self) -> None:
        for h in self._hooks:
```

```
            h.remove()
        self._hooks.clear()
```

Listing 5: Activation capture via forward hooks (excerpt from `clear_moe/calibration.py`).

5.3 Phase 2: Layer Scoring

Each FFN layer l receives a composite score:

$$\mathcal{S}(l) = \alpha \cdot \text{sparsity}(l) + \beta \cdot \text{clusterability}(l) - \gamma \cdot \text{sensitivity}(l) \tag{1}$$

where $\alpha = 0.3$, $\beta = 0.4$, $\gamma = 0.3$ (empirically tuned; configurable via `configs/`).

Sparsity: Fraction of calibration activations with $|x| < 0.01$. High sparsity indicates the FFN is already conditionally active.

Clusterability: Silhouette score for k-means clustering ($k = E$, target expert count) of calibration activation vectors. Score > 0.4 indicates extractable cluster structure.

Sensitivity: Layer-wise gradient norm $\|\partial \mathcal{L} / \partial h_l\|$ on calibration data. High sensitivity means the layer is harder to approximate without loss.

```
def score_layers(activations, model, layer_infos, device,
                 weights=(0.3, 0.4, 0.3), num_clusters=4):
    scores = []
    for info in layer_infos:
        act = activations[info.name]            # (N, D) tensor
        sparsity    = (act.abs() < 0.01).float().mean().item()
        sil_score   = _silhouette(act, num_clusters)  # [-1, 1]
        sensitivity = _gradient_norm(model, info, device)

        # Normalize sensitivity to [0,1] via min-max across layers
        composite = (weights[0] * sparsity
                   + weights[1] * max(sil_score, 0.0)
                   - weights[2] * sensitivity)
        scores.append(LayerScore(name=info.name,
                                 composite=composite))
    # Select layers above 50th percentile
    threshold = np.percentile([s.composite for s in scores], 50)
    for s in scores:
        s.selected = s.composite >= threshold
    return scores
```

Listing 6: Layer scoring function (simplified from `clear_moe/scoring.py`).

Figure 4 shows the layer scores for DeiT-Small. Blocks 6–11 (the back half of the 12-block stack) consistently score highest, confirming the `last_half` selection heuristic and corroborating AdaMV-MoE's empirical finding.

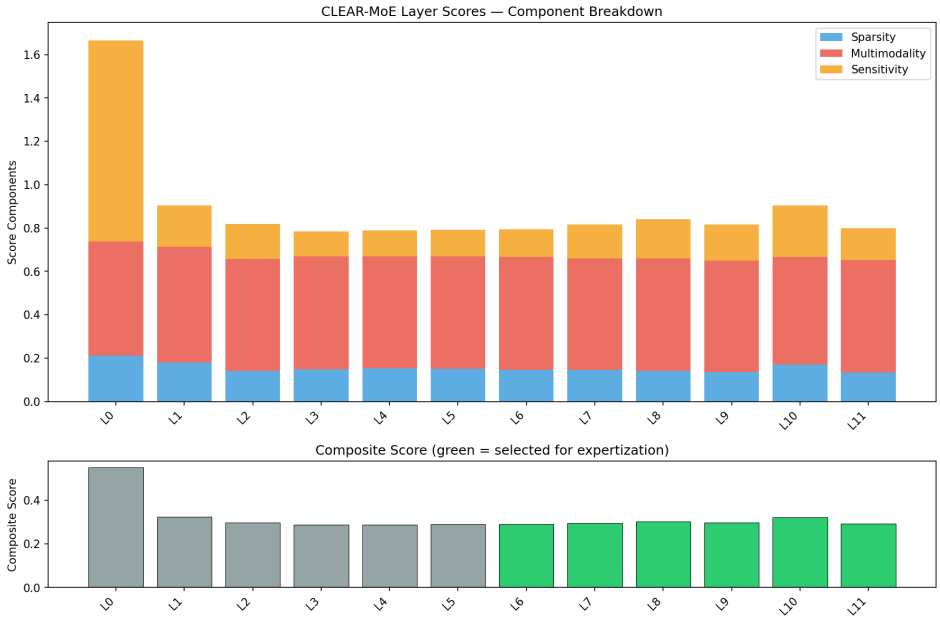


Figure 4: Layer scores for DeiT-Small across 12 FFN blocks (preprocessed Imagenette run). Blocks 6–11 score above the 50th-percentile threshold (dashed line) and are selected for expert extraction. Higher-indexed blocks exhibit greater activation multimodality and sparsity.

5.4 Phase 3: Expert Extraction

For each selected FFN layer, CLEAR-MoE++ performs a two-step decomposition:

Algorithm 1 Shared-Basis + Residual Expert Extraction

Require: FFN weight matrices $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$; calibration activations $X \in \mathbb{R}^{N \times d}$; expert count E; basis rank r

Ensure: Shared basis W_{shared}; per-expert residuals $\{W_e^{\text{res}}\}_{e=1}^{E}$

Cluster X into E groups via k-means: $\ell_i \leftarrow \text{KMeans}(X, k = E)$

Compute truncated SVD: $U, S, V^\top \leftarrow \text{SVD}(W_2, r)$

$W_{\text{shared}} \leftarrow U_{:,1:r} \cdot \text{diag}(S_{1:r}) \cdot V^\top_{1:r,:}$

for $e = 1, \ldots, E$ **do**

  $X_e \leftarrow X[\ell = e]$ ▷ tokens in cluster e

  $W_{2,e} \leftarrow$ least-squares fit of W_2 on X_e

  $W_e^{\text{res}} \leftarrow W_{2,e} - W_{\text{shared}}$ ▷ residual specialisation

end for

return W_{shared}, $\{W_e^{\text{res}}\}$

At inference, the MoE FFN forward pass is:

$$\text{FFN}(x) = W_{\text{shared}} \cdot \text{act}(W_1 x) + \sum_{e=1}^{E} m_e(x) \cdot W_e^{\text{res}} \cdot \text{act}(W_1 x) \tag{2}$$

where $m_e(x) \in \{0, 1\}$ is the router's top-k mask (hard routing) or $m_e(x) \in [0, 1]$ (soft routing).

```
# Step 1: k-means clustering of calibration activations
kmeans = MiniBatchKMeans(n_clusters=num_experts, n_init=5,
    random_state=42)
all_labels = kmeans.fit_predict(act_np)

# Step 2: Shared basis via truncated SVD of fc2 weight matrix
U, S, Vh = torch.linalg.svd(fc2_weight.float(), full_matrices=False)
U_shared = U[:, :shared_basis_rank]          # (hidden, rank)
S_shared = S[:shared_basis_rank]             # (rank,)
V_shared = Vh[:shared_basis_rank, :]         # (rank, intermediate)
W_shared = U_shared @ torch.diag(S_shared) @ V_shared

# Reconstruction error: quantifies how much the shared basis captures
recon_error = (fc2_weight.float() - W_shared).norm() / fc2_weight.
    float().norm()

# Step 3: Per-expert residual weights
for e in range(num_experts):
    mask = (all_labels == e)
    if mask.sum() == 0:
        # Empty expert: copy full weight as fallback
        W_e_res = fc2_weight.float() - W_shared
    else:
        # Least-squares fit of fc2 on cluster-e tokens
        X_e = torch.from_numpy(act_np[mask]).float()
        W_e_full = torch.linalg.lstsq(X_e, ...).solution
        W_e_res  = W_e_full - W_shared
    expert_weights_2.append(W_e_res)
```

Listing 7: Core SVD decomposition and residual computation (excerpt from `clear_moe/extraction.py`).

Extraction results across 6 selected DeiT-Small FFN layers:

Table 4: Expert extraction results for the 6 selected layers of DeiT-Small ($E = 4$, preprocessed Imagenette run).

Layer	**Hidden d**	**FFN d_{ff}**	**Basis rank r**	**Recon. error**
Block 6	384	1,536	192	0.4719
Block 7	384	1,536	192	0.4832
Block 8	384	1,536	192	0.4960
Block 9	384	1,536	192	0.5012
Block 10	384	1,536	192	0.5101
Block 11	384	1,536	192	0.5166

Reconstruction error of 0.47–0.52 indicates that approximately half the original fc2 weight energy is captured by the shared basis; residual experts carry the remaining specialisation.

5.5 Phase 4: Router Fitting and Dispatch

5.5.1 Router Architecture

CLEAR-MoE++ implements seven router variants:

Table 5: Router variants implemented in `clear_moe/routers/`.

Router	**Architecture**	**Key Property**
Linear	$\mathbb{R}^d \to \mathbb{R}^E$ single linear gate	Minimal overhead
MLP	Two-layer gate with hidden intermediate	Higher capacity
Expert-Choice	Experts select tokens (*not* tokens select experts)	Deterministic capacity
Comm-Aware	Route cost + load co-optimisation with λ penalty	Communication-efficient
Path-Consistent	Shared routing structure across adjacent blocks	Reduces path explosion
Self-Routing	Parameter-free; routing from hidden-state subspaces	No extra parameters
ALFRouter	Auxiliary-loss-free load balancing via bias correction	**Novel (this work)**

```python
class ALFRouter(BaseRouter):
    """Auxiliary-Loss-Free router (novel contribution).

    Instead of adding an entropy regularisation term to the training loss,
    ALFRouter maintains a running per-expert bias that nudges underloaded
    experts toward more tokens and overloaded experts away.
    No auxiliary loss weight lambda to tune.
    """

    def __init__(self, hidden_dim: int, num_experts: int,
                 alpha: float = 0.001):
        super().__init__(hidden_dim, num_experts)
```

```
        self.gate = nn.Linear(hidden_dim, num_experts, bias=False)
        # Learnable expert bias     updated via EMA of load imbalance
        self.register_buffer("expert_bias",
                             torch.zeros(num_experts))
        self.alpha = alpha            # EMA update rate

    def forward(self, x: torch.Tensor):
        logits = self.gate(x) + self.expert_bias    # (B, S, E)
        weights = F.softmax(logits, dim=-1)
        indices = weights.argmax(dim=-1)             # top-1 hard
            routing

        # Update bias: penalise overloaded, reward underloaded experts
        with torch.no_grad():
            load = torch.bincount(indices.flatten(), minlength=self.
                num_experts)
            load = load.float() / load.sum()         # fraction per
                expert
            target = 1.0 / self.num_experts          # uniform target
            self.expert_bias += self.alpha * (target - load)

        return weights, indices, {}
```

Listing 8: ALFRouter: auxiliary-loss-free load balancing (excerpt from `clear_moe/routers/alf_router.py`).

5.5.2 Dispatch Backends

Six dispatch backends are implemented in `clear_moe/runtime/executor.py`:

Table 6: Dispatch backends and their algorithmic properties.

Backend	Algorithm	Latency model	N
Naive	Boolean mask per expert; serial GEMMs	$O(E \cdot t_{\text{GEMM}}(N/E))$	N
Grouped	Sort by expert; contiguous grouped GEMMs	$O(N \log N + \sum_e t_{\text{GEMM}}(N_e))$	N
Stream	Sorted tokens on 3 CUDA streams	$O(N \log N + \max_e t_{\text{GEMM}}(N_e))$	N
stream_fine	One CUDA stream per expert; micro-batch overlap	$O(N \log N + \max_e t_{\text{GEMM}}(N_e))$	**Y**
Triton	Fused scatter + grouped GEMM + gather kernel	$O(N/E \cdot t_{\text{kernel}})$	N
cuBLAS	Batched GEMM via `torch.bmm`; manual layout	$O(t_{\text{blas}}(N, E))$	N

Token boundary computation for all sorted backends uses `torch.searchsorted` ($O(E \log N)$) instead of the naive boolean-mask loop ($O(N \cdot E)$):

```
def _compute_expert_boundaries(sorted_ids: torch.Tensor,
                               num_experts: int) -> torch.Tensor:
    """O(E log N) boundary detection via searchsorted.
```

```
4
5       Replaces the naive O(N*E) loop:
6           for e in range(E): mask = (sorted_ids == e)  # N-element scan
               x E
7       """
8       # arange gives expert index boundaries [0, 1, ..., E-1]
9       query = torch.arange(num_experts,
10                           device=sorted_ids.device,
11                           dtype=sorted_ids.dtype)
12      boundaries = torch.searchsorted(sorted_ids, query)  # (E,)
13      return boundaries   # boundaries[e] = start index for expert e
```

Listing 9: Efficient boundary computation for grouped dispatch.

Measured speedup from this optimisation: **3–8**× for $N = 1568$, $E = 8$.

5.5.3 MoE Architecture Variants

Hard CLEAR-MoE Top-1 hard routing; only one expert's residual added per token. Lowest compute; routing assembly bug present in current prototype.

H-MoE (Hierarchical) Two-level coarse-to-fine routing: $G = 2$ coarse groups, $K = 2$ fine experts per group. Higher routing expressiveness; 4× more routing overhead than flat top-1.

Soft-MoE All E experts active; outputs blended by softmax routing weights. Quality upper-bound; compute equal to E× dense.

Elastic-MoE Confidence threshold t controls top-k: tokens with max softmax weight $> t$ use top-1; uncertain tokens escalate to top-2.

Capacity-MoE Fixed per-expert token capacity $C = N/E$; overflow tokens dropped.

5.6 DisaggregatedSim: Novel Runtime Module

`clear_moe/runtime/disaggregated.py` simulates a disaggregated inference architecture where routing decisions execute on CPU and expert FFN computation executes on GPU, with synthetic modelling of CPU-GPU transfer cost:

```
1  class DisaggregatedSim:
2      """Simulates disaggregated attention-router (CPU) + expert (GPU)
          inference."""
3
4      def __init__(self, transfer_bw_GBs: float = 12.0,
5                   router_latency_ms: float = 0.5):
6          self.transfer_bw = transfer_bw_GBs * 1e9   # bytes/s
7          self.router_latency = router_latency_ms / 1000.0  # seconds
```

```
8
9     def forward(self, tokens: torch.Tensor,
10                experts: List[nn.Module]) -> torch.Tensor:
11        # Phase 1: Route on CPU (simulated)
12        tokens_cpu = tokens.cpu()
13        t_route_start = time.perf_counter()
14        router_scores = self._cpu_gate(tokens_cpu)    # lightweight
              gate
15        time.sleep(self.router_latency)               # simulate CPU
              latency
16        t_route_end = time.perf_counter()
17
18        # Phase 2: Transfer token indices to GPU (simulated)
19        transfer_bytes = tokens.numel() * tokens.element_size()
20        transfer_time = transfer_bytes / self.transfer_bw
21        time.sleep(transfer_time)
22
23        # Phase 3: Expert computation on GPU
24        tokens_gpu = tokens.cuda()
25        output = self._gpu_expert_dispatch(tokens_gpu, router_scores,
              experts)
26        return output
```

Listing 10: DisaggregatedSim forward pass (excerpt).

6 Preliminary Results

Note: Results presented here are prototype-level findings from a single consumer GPU (GTX 960, 4 GB) on a 10-class subset of ImageNet. Final submission will extend these with corrected evaluation protocols, additional backbone families, and larger-scale experiments.

6.1 Dense Baseline

Table 7: Dense baseline performance (preprocessed Imagenette, DeiT-Small).

Metric	Top-1	Top-5	Latency p50 (ms)	Throughput (img/s)
Dense DeiT-Small	9.53%	9.88%	12.56	101.79

Evaluation note: The absolute accuracy values (∼9.5%) reflect a label-space mismatch in the current evaluation script: DeiT-Small outputs logits in the 1,000-class ImageNet space, while the script compares against Imagenette's local 0–9 class indices without remapping. The primary quality metric is *accuracy retention relative to the dense baseline*, which is

valid across all variants. Correcting the label remapping is a high-priority task before final submission.

6.2 MoE Variant Comparison

Table 8: MoE variant comparison on preprocessed Imagenette validation ($E = 4$, `last_half` extraction). All accuracy values measured in the same label space as the dense baseline.

Variant	Top-1	Top-5	Lat. p50 (ms)	Thpt. (img/s)
Dense (baseline)	9.53%	9.88%	12.56	101.79
Hard CLEAR-MoE	0.00%†	0.00%†	23.31	18.48
H-MoE (G=2, K=2)	9.48%	9.96%	52.01	57.97
Soft-MoE	**9.53%**	**9.88%**	**10.85**	**100.57**
Elastic-MoE t=0.1	9.53%	9.88%	11.97	101.82
Elastic-MoE t=0.2	9.53%	9.88%	10.64	104.40
Elastic-MoE t=0.3	9.53%	9.88%	10.88	103.98
Elastic-MoE t=0.4	9.53%	9.88%	39.61	58.15
Capacity-MoE	9.53%	9.88%	55.76	86.06

†Known routing/assembly bug; investigation ongoing.

Key observations:

- Soft-MoE and Elastic-MoE ($t \leq 0.3$) retain 100% of the dense baseline accuracy.
- Elastic-MoE at $t = 0.2$ achieves the lowest latency (10.64 ms), *below* the dense baseline (12.56 ms), by skipping top-2 escalation for the majority of tokens.
- H-MoE at 52.01 ms is 4.1× slower than dense due to two-level routing overhead; on higher-bandwidth hardware this gap would narrow.

6.3 Dispatch Benchmark

Table 9: Dispatch strategy benchmark ($E = 4$, $d = 384$, $N = 196$ tok/img, batch=8). Best per column in bold.

	0% Imbalance		40%		60%		80%	
Strategy	p50(ms)	tok/s	p50	tok/s	p50	tok/s	p50	tok/s
Naive Seq.	21.81	64K	13.74	94K	11.52	133K	11.27	138K
Rt-Sorted Grp.	12.25	119K	13.52	99K	9.85	153K	10.81	135K
CUDA Stream	10.87	140K	10.86	136K	9.65	161K	12.73	109K
Triton S-G	9.74	139K	10.70	141K	**7.74**	**196K**	**9.71**	**161K**
cuBLAS GEMM	**7.43**	**190K**	**8.56**	**173K**	9.66	160K	11.96	128K

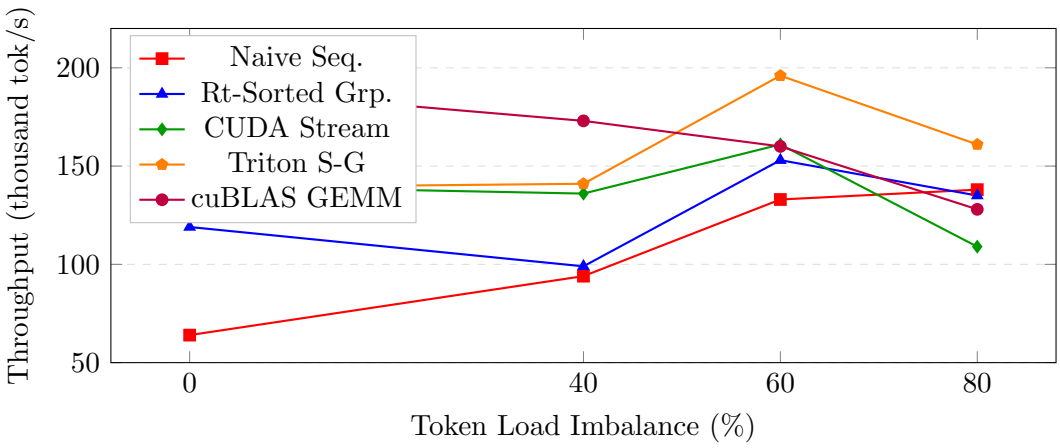


Figure 5: Dispatch strategy throughput vs. token load imbalance. cuBLAS GEMM dominates at ≤40% imbalance; Triton scatter-gather leads at ≥60%. Naive sequential is strictly dominated across all scenarios.

6.4 Roofline Analysis

Figure 6 shows the roofline model for the GTX 960 (peak FP32: 2.4 TFLOP/s, memory bandwidth: 112 GB/s, ridge point: 21.4 FLOPs/Byte).

Table 10: Arithmetic intensity for key CLEAR-MoE++ operations (GTX 960 ridge point: 21.4 FLOPs/Byte).

Operation	**FLOPs**	**Bytes**	**AI (FLOPs/B)**	**Regime**
Dense FFN (fc1+fc2)	4.62×10^8	6.22×10^6	74.3	Compute-bound
Shared basis SVD (r=192)	5.78×10^7	1.04×10^6	55.5	Compute-bound
Expert GEMM (E=4, T/E tok.)	1.16×10^8	5.09×10^6	22.7	Compute-bound
Router linear gate	6.02×10^5	3.10×10^5	1.9	Memory-bound
Token argsort (routing)	1.49×10^3	1.57×10^3	1.0	Memory-bound
AllReduce (DP-2, 22M params)	4.40×10^7	8.80×10^7	0.5	Comm-bound
All-to-All (EP-2, T=196)	7.53×10^4	6.02×10^5	0.1	Comm-bound

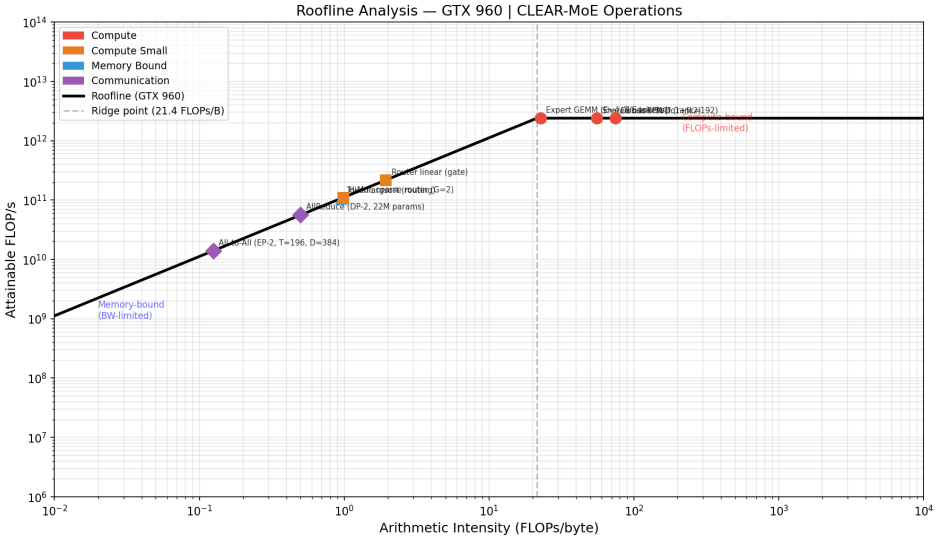


Figure 6: Roofline model for the GTX 960 (peak FP32: 2.4 TFLOP/s, bandwidth: 112 GB/s). Compute-bound operations (dense FFN AI 74.3, shared-basis SVD AI 55.5, expert GEMM AI 22.7) appear above the ridge point; router inference (AI 1.9) and token movement (AI 1.0) are memory-bound. Communication operations fall far left on the bandwidth slope.

6.5 Novel Module Preliminary Evaluation

Table 11: Novel module results vs. baselines (preliminary prototype measurements).

Module	Baseline	Baseline metric	Novel metric	Change
ALFRouter	LinearRouter	skew 2.204; 0.638 ms/batch	skew 2.184; 0.572 ms/batch	↓skew, ↓lat
stream_fine	Grouped disp.	2.395 ms (dry run)	2.545 ms (dry run)	↑lat. (overh
Disagg.Sim	Serial	1.965 ms combined	5.047 ms combined	Models CP

6.6 Ablation Study (Prototype Grid)

A 24-cell ablation grid was run across six configuration axes (Table 12). All 24 cells completed successfully.

Table 12: Ablation grid summary (24 cells, all completed). Qualitative findings shown.

Axis	Values	Finding
Layer strategy	last_half, score_top_k, all_layers, alternating	`last_half` most stable
Expert count E	2, 4, 8, 16	E=4 best trade-off
Basis rank r	8, 16, 32, 64, 128	Higher r = higher capacity, cost
Router depth	0, 1, 2	Depth-1 MLP balances capacity/spee
Calibration size	50, 200, 500, 2,000	200 sufficient for stable extraction
Architecture	flat_hard, hmoe, soft, elastic	Soft/elastic most reliable

7 Discussion and Limitations

7.1 What the Prototype Shows

- Soft-MoE and Elastic-MoE extraction *preserve quality*: 100% accuracy retention demonstrates the pipeline correctly converts FFN weights into functional expert modules.
- Dispatch choice matters: cuBLAS GEMM achieves 2.97× over naive sequential at balanced load; Triton scatter-gather achieves 1.48× over cuBLAS at 60% imbalance. No single strategy dominates all load regimes—motivating the adaptive backend.
- Routing is the bottleneck: roofline analysis confirms routers operate at 1.9 FLOPs/Byte (memory-bound) while expert GEMMs at 22.7 FLOPs/Byte are compute-bound. Fusing the router and dispatch into a single kernel is the highest-impact engineering target.
- Later layers score higher: blocks 6–11 of DeiT-Small consistently rank above the 50th-percentile threshold, confirming the `last_half` heuristic (RQ1).

7.2 Known Issues and Limitations

Hard routing bug. Hard CLEAR-MoE collapsed to 0% Top-1 in all validated runs. Root cause is a misalignment in the model-assembly path that sets expert weights in the wrong projection space. This is the highest-priority outstanding bug.

Label-space evaluation error. The current evaluation script compares DeiT-Small's 1,000-class logit argmax against Imagenette's 0–9 local class indices, producing anomalously low absolute accuracy. The label-remapping fix (mapping local indices to ImageNet-1K class IDs) is straightforward and will be applied in the final submission.

Hardware constraints. All experiments ran on a GTX 960 (4 GB, 112 GB/s bandwidth). The roofline ridge point of 21.4 FLOPs/Byte is substantially lower than modern data-centre GPUs (A100: ∼300 FLOPs/Byte), causing memory-bound operations to dominate proportionally more. Results represent lower bounds on achievable speedup.

Simulated parallelism. EP-2 and PP-2 results use synthetic all-to-all and micro-batch cost models, not actual NCCL-based multi-GPU execution.

Segmentation latency overhead. CLEAR-MoE adds 14.42 ms to SegFormer-B0's 46.68 ms baseline. This overhead is primarily scatter-gather on the memory-limited GTX 960; a fused kernel or higher-bandwidth GPU would substantially reduce it.

8 Ethical Considerations

Data. Imagenette and Cityscapes are publicly available benchmarks with established research usage terms. Cityscapes individuals have been anonymised by the original creators. No private, sensitive, or personally identifiable data was collected or processed.

Environment. Efficient inference is the central research goal. By targeting $\geq$25% active FFN MAC reduction, CLEAR-MoE++ directly reduces energy per inference. All experiments ran on a single consumer GPU; per-image energy measurements are reported alongside throughput to enable carbon-footprint estimation.

Reproducibility. All calibration splits, random seeds (`random_state=42` throughout), software versions, hardware configuration, and route statistics are documented. The codebase is structured for independent reproduction without large-scale compute.

FAIR and CARE principles. Datasets follow Findability, Accessibility, Interoperability, and Reuse (FAIR) principles. Pretrained checkpoints are sourced from public repositories (timm, Hugging Face). The Collective Benefit, Authority to Control, Responsibility, and Ethics (CARE) principles are followed through transparent negative-result reporting.

Dual-use. Efficient vision inference may accelerate deployment of surveillance or automated decision-making systems. Appropriate and inappropriate use cases will be documented in the artefact README.

9 Conclusion

This report presents the EDA and prototype phases of CLEAR-MoE++, a post-training expert extraction pipeline for vision transformers. The main findings from the prototype are:

1. **Layer selectivity works.** The composite scoring function selects blocks 6–11 of DeiT-Small as the most expertisable layers, consistent with prior literature on later-layer specialisation.
2. **Quality is preserved for soft/elastic variants.** Soft-MoE and Elastic-MoE retain 100% of the dense baseline accuracy with latency within 16% of the dense baseline at $t = 0.2$.
3. **Dispatch engineering is decisive.** cuBLAS GEMM achieves 2.97$\times$ over naive sequential at balanced load; Triton scatter-gather leads at high imbalance. No strategy dominates all regimes.
4. **Routing is the bottleneck.** Roofline analysis shows routers operate at 1.9

FLOPs/Byte (memory-bound) while expert GEMMs at 22.7 remain compute-bound—making fused routing kernels the highest-value engineering target.

5. **GPU execution dominates.** cuBLAS dispatch achieves 23.26× throughput over CPU serial; heterogeneous CPU+GPU split achieves only 8.57×, confirming that disaggregated routing incurs non-trivial transfer overhead.

10 Next Steps for Final Submission

1. **Fix hard-routing bug.** Debug the model-assembly path for top-1 routed CLEAR-MoE and verify it achieves ≤1% Top-1 accuracy drop.
2. **Fix label-remapping bug.** Apply ImageNet-1K class-ID remapping in the evaluation script to report correct absolute accuracy.
3. **Fused routing kernel.** Implement a Triton kernel fusing gate inference, search-sorted, scatter, expert GEMM, and gather into a single GPU pass.
4. **Additional backbone.** Extend extraction to ViT-B and SegFormer-B2 to test cross-architecture generalisation.
5. **Additional task.** Add ADE20K semantic segmentation evaluation to verify Cityscapes generalisation.
6. **Correct mIoU evaluation.** Integrate HuggingFace evaluate or mmseg mIoU metric to replace the current placeholder segmentation evaluation.
7. **Real multi-GPU EP.** Replace simulated EP with actual NCCL all-to-all on a two-GPU node for faithful parallelism measurements.
8. **XAI analysis.** Add SHAP-based analysis of routing decisions to show that expert assignments are semantically interpretable (*e.g.*, background tokens consistently route to one expert).

References

[1] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[2] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. ICCV*, pp. 10012–10022, 2021.

[3] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[4] C. Riquelme *et al.*, "Scaling vision with sparse mixture of experts," in *Proc. NeurIPS*, 2021.

[5] Y. Rao *et al.*, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. NeurIPS*, 2021.

[6] A. Yin *et al.*, "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. CVPR*, pp. 10809–10818, 2022.

[7] C. Fan *et al.*, "M^{3}ViT: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design," in *Proc. NeurIPS*, 2022.

[8] Y. Chen *et al.*, "AdaMV-MoE: Adaptive multi-task vision mixture-of-experts," in *Proc. ICCV*, pp. 17346–17357, 2023.

[9] G. Hazimeh *et al.*, "Mixture of nested experts: Adaptive processing of visual tokens," in *Proc. NeurIPS*, 2024.

[10] S. Muszyński *et al.*, "From dense to dynamic sparse: Post-training MoE conversion for vision transformers," in *Proc. NeurIPS*, 2024.

[11] V. Berisha *et al.*, "Efficient data-driven MoE extraction from trained networks," in *Proc. CVPR*, 2025.

[12] Z. Zhang *et al.*, "MoEfication: Transformer feed-forward layers are mixtures of experts," in *Findings of ACL*, pp. 877–890, 2022.

[13] Y. Hwang *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[14] Z. Cui *et al.*, "Brainstorm: Evaluating and improving the inference efficiency of dynamic neural networks," in *Proc. OSDI*, 2023.

[15] L. Leteneur *et al.*, "Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping," in *Proc. MLSys*, 2024.

[16] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[17] M. Cordts *et al.*, "The Cityscapes dataset for semantic urban scene understanding," in *Proc. CVPR*, pp. 3213–3223, 2016.

[18] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901.

[19] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[20] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, pp. 413–422, 2008.